A Compute-Aware Benchmark of POMDP Solvers: Performance–Compute Trade-offs Across Environments

Ioan Hedea
Delft University of Technology (TU Delft)

April 10, 2026

Abstract

We present a controlled benchmark of classical and modern POMDP solvers, spanning exact, point-based, and online simulation methods. The benchmark evaluates solver behavior across standard and custom environments with matched computational budgets and reports discounted return, runtime per step, total online compute per episode, and belief-divergence estimates for particle-based methods. A consistent pattern emerges: PBVI dominates the small structured domains, while online methods become more competitive in the harder settings, but at much higher online compute cost. To interpret that crossover, we formulate a compute-allocation view of solvers and advance a testable structural conjecture, formalized through a proxy complexity score, that environments with compact reusable belief structure favor offline preallocation whereas higher branching-adjusted uncertainty shifts the Pareto frontier toward adaptive online search. The benchmark also sharpens an important cautionary point: even with 100 episodes per setting, the merge environments retain very wide confidence intervals, so solver comparisons there remain statistically fragile. As a secondary method study, we instantiate the compute-allocation view as `BASDeepRollout`; in the learned setting it becomes competitive in the harder reactive-planning regime exemplified by DrivingMerge, but does not overturn the broader structure-dependent picture. Rather than supporting a single overall winner, the paper argues that solver choice is fundamentally a performance–compute trade-off conditioned on environment structure.

1 Introduction

1.1 Motivation

Sequential decision-making under uncertainty is central to robotics, planning, diagnosis, and safety-critical AI. POMDPs provide a principled formulation [2], but practical solver behavior strongly depends on environment structure, belief representation, and available compute.

1.2 Problem

Most empirical comparisons still suffer from at least one of three limitations: they focus on a single benchmark, report reward without compute, or omit enough protocol detail to make reproduction difficult. This is especially problematic for POMDPs, where the best-performing solver in one domain may be impractical in another once online latency is taken seriously. Our goal in this paper is therefore benchmark-first: not to argue for a new universal solver, but to compare solver regimes under matched compute and shared implementations, and to ask whether environment structure helps explain the observed offline–online crossover.

1.3 Contributions

We provide:

- Joint reporting of return, online compute, and belief-quality diagnostics, so solver comparison is framed as a performance–compute trade-off rather than a reward-only leaderboard.
- A multi-environment benchmark spanning both canonical and custom domains, including harder settings that expose crossover points between offline and online solver classes.
- A reproducible benchmarking harness with saved artifacts, consistent seed control, and explicit skip handling for unsupported configurations.
- A structural crossover conjecture, together with a simple proxy score, for interpreting when reusable offline structure should dominate adaptive online search.
- Empirical analysis of failure modes and budget non-monotonicity, including a persistent POMCP failure on Tiger and regimes where more computation does not buy better return.
- A secondary compute-allocation method study showing that deeper allocation, rollout gating, and leaf-value bootstrapping can help in harder online-planning regimes, but do not yet support a universal method claim.

2 Background

2.1 POMDP Formulation

A POMDP is defined as

$$\mathcal{P} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, T, Z, R, \gamma, b_0), \tag{1}$$

where $\mathcal{S}$ is the state set, $\mathcal{A}$ actions, $\mathcal{O}$ observations, $T(s' \mid s, a)$ transition dynamics, $Z(o \mid s', a)$ observation model, $R(s, a)$ reward, $\gamma \in (0, 1)$ discount factor, and b_0 the initial belief. Belief updates follow Bayes filtering:

$$b'(s') \propto Z(o \mid s', a) \sum_{s \in \mathcal{S}} T(s' \mid s, a) b(s). \tag{2}$$

Finite-horizon exact dynamic programming for POMDPs follows the classical piecewise-linear convex structure [11, 5].

2.2 Solver Categories

We evaluate methods from four categories:

- **Exact:** finite-horizon tabular value iteration with piecewise-linear convex value representations.
- **Offline approximate:** PBVI with sampled belief points [7].
- **Online simulation:** POMCP and a DESPOT-style sparse-scenario planner [10, 12].
- **Adaptive online variant:** AdaOPS-style entropy-adaptive simulation budgeting.

2.3 Solver as a Budget-Constrained Policy Mapping

A POMDP solver can be viewed as a mapping from a problem instance and a computational budget to a policy. Let $P = (\mathcal{S}, \mathcal{A}, \mathcal{O}, T, Z, R, \gamma, b_0)$ denote a POMDP, and let B denote a computational budget such as the number of belief points, particles, scenarios, or simulations. A solver then induces a policy

$$\pi_B = \mathcal{S}(P, B). \tag{3}$$

An even more structural view is to treat the solver as a compute-allocation policy over reachable belief space. Let $\mathcal{B}_{\text{reach}}(P)$ denote the set of beliefs reachable under the POMDP dynamics. A solver with budget B induces an allocation rule

$$a_B : \mathcal{B}_{\text{reach}}(P) \to \mathbb{R}_{\geq 0}, \quad \int_{\mathcal{B}_{\text{reach}}(P)} a_B(b)\, db \leq B, \tag{4}$$

which describes where computation is spent. The resulting policy can then be viewed as

$$\pi_B = \Gamma(P, a_B), \tag{5}$$

for some planner-dependent policy extraction operator Γ. Offline methods preallocate computation and reuse it at run time, online methods adapt allocation around the current belief, and adaptive variants modulate that spending using uncertainty statistics. This framing shifts the comparison from named solvers to compute-allocation strategies, evaluated jointly by expected return $R(\pi_B)$ and compute cost $C(\pi_B)$:

$$\max_{\mathcal{S}, B} \quad R(\pi_B) \quad \text{s.t.} \quad C(\pi_B) \leq \kappa, \tag{6}$$

for a deployment-specific compute allowance κ.

2.4 A Compute-Allocation Method Instantiation

The same abstraction can also be instantiated as a concrete online planner. In the method study below, we use a belief-adaptive search variant called `BASDeepRollout`. It modifies a POMCP-style planner in two places. First, it replaces uniform action priors with a belief-conditioned allocation rule

$$p_\theta(a \mid b, d) = \text{Normalize}((1 - \lambda_d) u(a) + \lambda_d\, \phi_\theta(a, b)), \tag{7}$$

where $u(a)$ is the uniform prior over actions and $\lambda_d = \lambda_0 \rho^d$ decays the allocation strength with tree depth d. In the implemented method,

$$\begin{aligned} \phi_\theta(a, b) = {} & w_r \bar{Q}_1(a, b) + w_s \bar{H}_S(a, b) \\ & + w_o \bar{H}_O(a, b) + \eta_{\text{env}}(a, b), \end{aligned} \tag{8}$$

where $\bar{Q}_1(a, b)$ is a normalized one-step reward proxy, $\bar{H}_S(a, b)$ and $\bar{H}_O(a, b)$ are normalized predictive state and observation entropies, and $\eta_{\text{env}}(a, b)$ is a lightweight environment-specific shaping term. We use $\rho = 0.75$, with $\lambda_0 = 0.12$ for RockSample-style domains and $\lambda_0 = 0.16$ for Tiger, the merge tasks, and MedicalDiagnosis, so the bias is strongest near the root and fades with depth.

Second, `BASDeepRollout` adapts rollout effort to belief difficulty using a guarded signal rather than raw entropy alone. Let

$$s(b) = g(P, b)\, (0.45\, \Delta(b) + 0.35\, V(b) + 0.20\, R(b)), \tag{9}$$

where $\Delta(b)$ is immediate action-value disagreement, $V(b)$ is a value-of-information proxy based on predictive observation entropy, and $R(b)$ is a catastrophe-risk term. The guardrail $g(P, b) \in [0, 1]$ is implemented as a clipped product of state-size, belief-support, and environment factors, so rollout effort is reduced on tiny or support-collapsed beliefs and in highly structured domains such as Tiger. Concretely, the implementation downweights very small state spaces (0.45 when $|\mathcal{S}| \leq 4$, 0.65 when $|\mathcal{S}| \leq 8$), sharp beliefs (support-size factors 0.60/0.80/1.0 and peak-belief factor 0.70 when $\max_s b(s) \geq 0.8$), and especially structured environments such as Tiger (0.55) and MedicalDiagnosis (0.85). The rollout horizon is then

$$H_{\text{roll}}(b) = H_{\min} + \text{round}(H_{\text{extra}}\, \alpha(P)\, s(b)), \tag{10}$$

with $H_{\min} = 3$, $H_{\text{extra}} = 4$, and environment scale $\alpha(P) \in \{0.55, 1.10, 1.05, 0.85\}$ for Tiger, RockSample, DrivingMerge, and MedicalDiagnosis respectively. Intuitively, the final leaf term is just a cheap two-step surrogate so truncated search does not back up an uninformative terminal value. The planner therefore uses a one-step leaf bootstrap

$$\hat{V}_{\text{leaf}}(b) = \max_a \Big[Q_1(a,b) + \gamma \sum_o p(o \mid b,a) \max_{a'} Q_1(a', \tau(b,a,o)) \Big], \tag{11}$$

so that shallow search still receives a belief-aware terminal estimate. The method therefore applies the compute-allocation view to action selection, rollout depth, and leaf evaluation. In the learned variant studied below, the same structure is retained but the action prior and leaf value are supplied by a compact policy–value network trained from teacher search traces, letting us separate the effect of compute allocation from the quality of the learned guidance driving it.

3 Related Work

Classical POMDP planning established the belief-state view and the piecewise-linear convex structure exploited by exact finite-horizon methods [2, 11, 5]. As exact methods became difficult to scale, point-based approximations such as PBVI and SARSOP provided practical offline compromises by restricting backups to sampled or optimally reachable beliefs [7, 4]. In parallel, online planning developed into a substantial line of work in its own right: AEMS and related heuristic-search methods explicitly refined approximate value functions online [8, 9], POMCP made Monte Carlo tree search practical in large discrete POMDPs [10], and DESPOT and IS-DESPOT improved sparse-scenario search and sampling efficiency [12, 6]. More recent variants such as POMCPOW extend Monte Carlo planning to continuous observation settings [13]. A separate thread embeds planning structure inside learned models or differentiable architectures, for example QMDP-Net and deep variational RL for partially observable control [3, 1].

Our goal is not to survey all modern partial-observability methods or to claim a new universal solver. Instead, we study a narrower question that is often obscured in prior comparisons: how solver families trade return against online compute when they are run through a shared harness, matched against the same environment implementations, and evaluated with explicit uncertainty reporting. For that reason, the benchmark centers classical planning regimes that can be compared cleanly under a common tabular model and common compute accounting. Learned-model methods remain important context for external validity, and POMCPOW marks an especially relevant next baseline once the suite includes more continuous-observation tasks, but neither is yet part of the main matched benchmark reported here.

4 Experimental Setup

4.1 Solvers

The benchmark suite includes Exact Value Iteration, PBVI, POMCP, DESPOT, an optional AdaOPS-style adaptive online variant, and an optional Julia bridge for SARSOP [4]. This selection is intentional: it emphasizes solver families that can be compared under a shared tabular model, a shared environment API, and a common notion of online compute. All methods are evaluated through the same Python harness, which lets us attribute observed differences to solver behavior rather than to inconsistent environment implementations. SARSOP is included for completeness, but it is not treated as a resolved headline baseline because its Julia/POMDPX toolchain did not execute robustly across the full suite; those failures are preserved as explicit `skipped` records in the raw artifacts rather than hidden from the report.

4.2 Environments

The environments are chosen to probe solver regimes rather than to maximize raw benchmark count.

Tiger. A canonical two-state listening/opening problem that stresses information gathering under sparse observations and severe wrong-door penalties.

RockSample. The 4x3 instance is a structured information-gathering task with movement, checking actions, and sampling decisions. It increases state and observation complexity relative to Tiger while preserving tabular tractability.

DrivingMerge (custom). A partially observable merge-control task with hidden gap size, noisy sensor readings, and actions `wait`, `slow_merge`, and `merge`. Rewards penalize collision and excessive waiting while incentivizing safe merge completion.

MedicalDiagnosis (custom). A sequential test-and-stop problem with latent health severity, costly information-gathering actions (blood test, imaging), and terminal diagnosis actions. It captures practical trade-offs among test cost, delay risk, and misdiagnosis penalties.

Harder optional settings. We additionally support a larger RockSample instance (`RockSample(5,4)`) and a noisier merge sensor variant (`DrivingMergeNoisy`) to stress scalability and robustness.

4.3 Belief Budget

We define *belief budget* as the primary family-specific compute knob used for matched comparison, mapped by solver type:

- POMCP/DESPOT: number of particles/scenarios.
- PBVI: number of belief points.
- Exact/SARSOP variants: budget-insensitive in online execution (single budget setting reported).

The goal is not to claim that these budgets are identical across solver families in every algorithmic sense, but to expose how each family trades off performance against compute as its main approximation budget changes. Evaluated values: $\{64, 128, 256, 512\}$.

4.4 Metrics

Performance:

- Discounted return (reported in the paper as mean ± 95% confidence interval)

Compute:

- Runtime per step (ms, reported as mean ± 95% confidence interval)
- Total compute per episode (s, reported as mean ± 95% confidence interval)

Belief quality (where available):

- Jensen–Shannon belief divergence between exact and particle beliefs

4.5 Protocol

Unless otherwise noted, we run 100 episodes per solver–environment–budget setting, using fixed seed generation from a shared base seed to ensure reproducibility and paired comparability. The extended study covers 102 solver–environment–budget combinations in total, of which 100 complete successfully and 2 are recorded as explicit skips because exact value iteration exceeds the RockSample state cap. All solvers are evaluated under the same environment dynamics and reward models. The primary benchmark claim rests on Tiger, RockSample(4,3), DrivingMerge, and MedicalDiagnosis; the larger RockSample(5,4) and noisier DrivingMergeNoisy variants are used as stress-test extensions to expose scalability and robustness effects beyond the core suite. Figures and headline tables report 95% confidence intervals, while the CSV artifacts retain standard deviations for downstream analysis.

4.6 Implementation Details and Reproducibility

The benchmark is implemented as a shared Python harness with solver adapters layered over a common tabular environment interface. Each run emits machine-readable CSV and JSON summaries, and all plots in the paper are regenerated from those saved artifacts rather than from ad hoc notebook state. Confidence intervals are computed as $1.96\,\sigma/\sqrt{n}$ with $n = 100$ episodes. To reduce hidden variance, solver evaluations use the same base-seed schedule, identical reward definitions, and identical environment dynamics. Optional components are handled explicitly rather than silently omitted: the Julia SARSOP bridge is isolated behind a separate adapter, and unsupported exact-planning configurations are recorded as `skipped` when the state cap is exceeded. This design makes the benchmark easier to audit, rerun, and extend, while also preserving negative integration results that would otherwise disappear from the paper narrative. Appendix Table 5 therefore exposes the best solver at every tested budget rather than only the headline winners.

Table 1: Benchmark design summary.

Environment	Pressure point	Why it matters
Tiger	Sparse information, severe penalties	Canonical sanity check
RockSample (4,3)	Exploration structure	Small tabular benchmark
RockSample (5,4)	Larger branching structure	Scaling stress test
DrivingMerge	Reactive uncertainty	Noisy-control task
DrivingMerge Noisy	Observation corruption	Robustness stress test
Medical diagnosis	Test-vs-stop trade-off	Costly sensing decisions

4.7 Benchmark Design Snapshot

Table 1 summarizes the role each environment plays in the study. This matters because the benchmark is not intended as a leaderboard over arbitrary tasks; it is meant to probe distinct solver failure modes. Tiger isolates information gathering under catastrophic penalties, RockSample emphasizes scalable information acquisition, the merge tasks stress reactive planning under noisy sensing, and MedicalDiagnosis tests whether a solver can trade off delayed information gain against irreversible terminal decisions.

5 Results

5.1 Scalability Curves

Figure 1 summarizes the expanded six-environment 300-episode budget sweep with `SARSOPJulia` included, while `BASDeepRolloutLearned` appears where learned checkpoints are available. The main signal is no longer a simple "small offline, large online" split. Instead, strong offline alpha-vector methods remain dominant across the structured tabular suite: `SARSOPJulia` or PBVI lead Tiger, both RockSample variants, and MedicalDiagnosis. The merge tasks remain the clearest exception. There PBVI stays strongly negative, while the best online or learned-search points move closer to zero and occasionally above it, but the confidence intervals are still wide enough that the ranking remains open. The learned-BAS overlay is still informative, but mainly diagnostically. Its broadly flat scaling where it is defined suggests that the current bottleneck is representational rather than computational: the learned planner receives broadly similar guidance regardless of how many simulations it is allowed to run, so

additional search cannot recover value that the model is not encoding.

5.2 Return–Compute Pareto Trade-off

Figure 2 complements the budget sweeps by showing return-versus-compute scatter plots with per-environment Pareto frontiers. Tiger, RockSample(4,3), MedicalDiagnosis, and now RockSample(5,4) all tell the same broader benchmark story: the strongest offline points sit far to the left and above the online methods, meaning they achieve higher return at dramatically lower online compute. With `SARSOPJulia` in the comparison, even the larger RockSample instance no longer looks like an online-planning win. That is an important update to the paper's structural framing. The merge tasks remain the more plausible online-favoring regime: PBVI is strongly negative there, while the strongest non-dominated frontier points come from DESPOT, AdaOPS, or learned BAS. Against that backdrop, `BASDeepRolloutLearned` is best read as a supporting probe: it does not replace strong offline baselines on structured tasks, but it becomes competitive exactly in the regime where offline structure reuse weakens.

5.3 Key Quantitative Snapshot

Table 2 summarizes best mean return per environment from the 300-episode expanded run (132 combinations total; 122 completed; 10 skipped, including exact-state caps on the RockSample variants and missing learned-BAS checkpoints for RockSample(5,4) and DrivingMergeNoisy). Rows marked *Open* remain statistically unresolved at the current sample count. The table should be read as a regime summary rather than as a definitive ranking: the most important signal is which solver family occupies the best-return row in each environment, together with whether that row is actually resolved.

5.4 Key Observations

Observation 1: No universal solver. No single method dominates all environments. As visible in Figure 1 and Table 2, `SARSOPJulia` or PBVI lead Tiger, both RockSample variants, and MedicalDiagnosis, while the merge tasks remain unresolved and much less favorable to offline structure reuse. The strongest mean on DrivingMerge comes from learned BAS at budget 128, while DrivingMergeNoisy is best on mean return under DESPOT at budget 128, but both remain statistically open.

Observation 2: Offline vs online trade-off. Offline alpha-vector methods are stronger than the earlier draft suggested. The Tiger, RockSample(4,3), MedicalDiagnosis, and RockSample(5,4) Pareto panels in Figure 2 all place the strongest offline points far to the left of the online methods while also matching or exceeding their return. The most important update is RockSample(5,4): once `SARSOPJulia` is included, the best result is no longer an online-planning win but an offline one (16.77 ± 0.64 at negligible online compute relative to the multi-second online planners). Table 3 still shows the counterweight to that story: low online latency can hide substantial one-time setup cost for offline planning, reaching about 15.85 seconds for PBVI on RockSample(4,3) at budget 512.

Observation 3: Compute–performance coupling. Increasing budget does not uniformly improve return; several solver–environment pairs show non-monotonic gains or early saturation. Compute, however, tends to rise reliably for online planners. This asymmetry is exactly why return-only evaluation is misleading: larger budgets can buy better performance, but sometimes only at a steep and practically unacceptable online cost.

Appendix Table 5 makes this concrete. On RockSample(5,4), for instance, POMCP is nearly flat in return across budgets 64–512 (roughly 8.6 to 9.0) while still consuming about 2.15–2.69 seconds of online compute per episode, and AdaOPS costs roughly 5.49–5.87 seconds per episode for returns clustered around 8.0–8.5. By contrast, the strongest offline points remain orders of magnitude cheaper online. That is exactly the kind of budget pathology that a reward-only benchmark would miss.

Observation 4: Environment structure matters. MedicalDiagnosis and both RockSample variants favor structured offline planning, where belief compression and reusable value structure are advantageous; this is visible both in Figure 1 and in the left-shifted `SARSOPJulia`/PBVI points of Figure 2. That is precisely why the new RockSample(5,4) result matters: larger state support alone is not enough to force an online-search crossover. The merge tasks appear to be the more plausible crossover candidates, because their useful beliefs seem less reusable and more timing-sensitive, but they remain statistically noisy even after 300 episodes.

Observation 5: Belief divergence trends are measurable. For particle-based solvers, Jensen–Shannon divergence between exact and particle beliefs is directly tracked, but it is not a simple proxy for control performance. Lower divergence usually means the particle approximation is becoming more faithful, yet the benchmark continues to show that accurate belief tracking is only one ingredient of good control. Hard environments can have relatively low divergence at large budgets while still exhibiting poor control because rollout quality, branching allocation, or horizon truncation, rather than belief approximation alone, becomes the bottleneck.

Observation 6: Tiger exposes an important POMCP failure mode. The strongest anomaly in

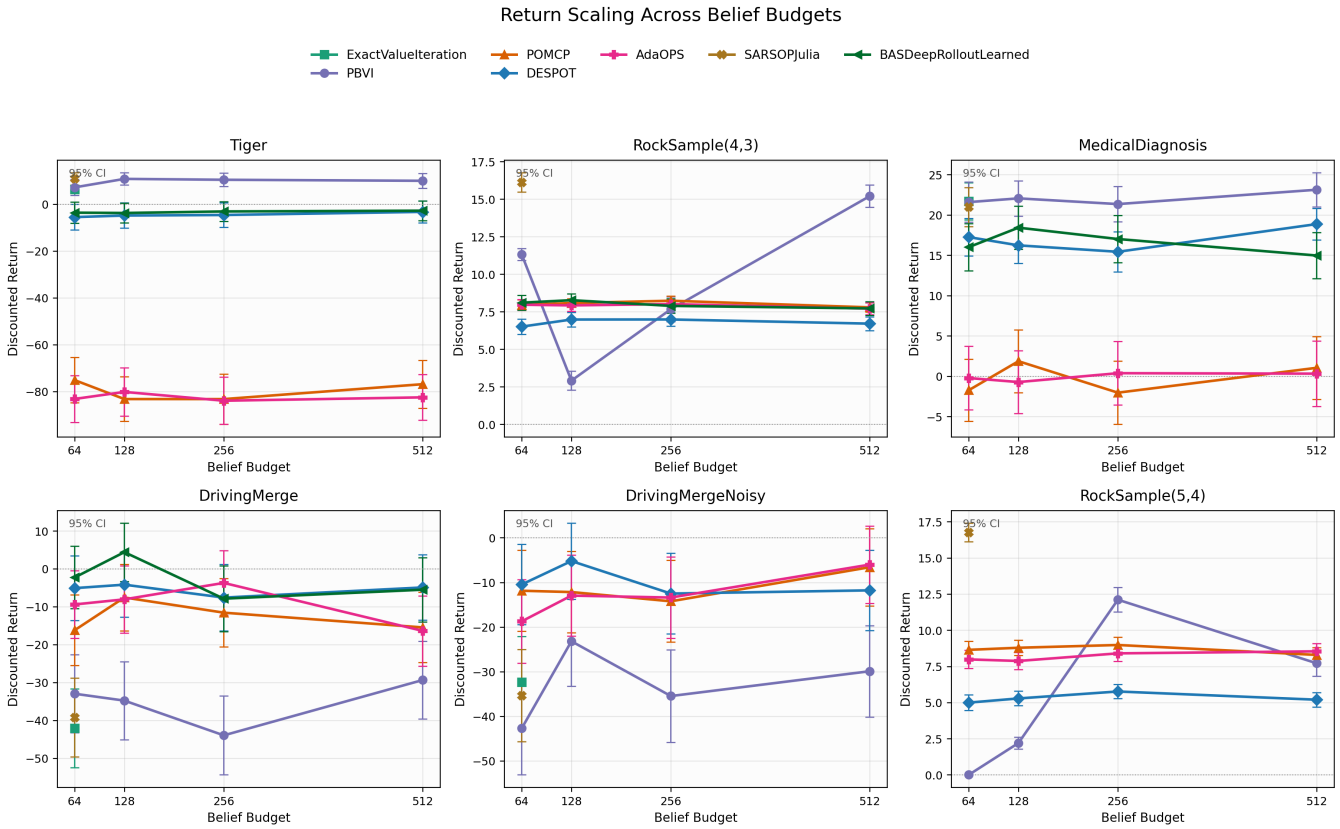

Figure 1: Return scaling from the 300-episode expanded benchmark with `SARSOPJulia` included and `BASDeepRolloutLearned` shown where learned checkpoints exist. Each panel reports mean discounted return with 95% confidence intervals for one environment and budgets 64/128/256/512 (budget-invariant solvers appear at a single point). The main benchmark pattern is that strong offline alpha-vector methods remain competitive much farther into the hard tabular regime than a raw size-based story would suggest; the learned-BAS curves remain secondary diagnostics rather than central benchmark winners.

the study remains POMCP's poor performance on Tiger, a canonical benchmark where one might expect online search to be competitive. In the Tiger panels of Figures 1 and 2, POMCP remains strongly negative across all tested budgets (roughly -75 to -83 mean return), while `SARSOPJulia` and PBVI remain positive. This failure therefore survives the larger-sample rerun rather than washing out with more episodes. In our setting the underperformance is consistent with a combination of factors: Tiger is tiny and highly structured, so offline methods can exploit its geometry almost perfectly; wrong-door actions are catastrophically costly; and Monte Carlo search with approximate particle beliefs and simple rollout policies can be brittle when a few poor branches dominate value estimates. Rather than dismissing this as a mere implementation artifact, we view it as a useful reminder that canonical benchmarks can still reveal where an online method is mismatched to the structure of the problem.

Observation 7: More episodes clarify, but do not fully resolve, the merge tasks. The additional episodes make the merge conclusions more honest. On DrivingMerge, the best observed mean is now learned BAS at budget 128 with 4.40 ± 7.70, while DrivingMergeNoisy is best on mean return under DESPOT at budget 128 with -5.23 ± 8.53. Those intervals are narrower than before, but they still overlap zero and several competing methods. That is why both merge rows are marked *Open* in Table 2, and why the DrivingMerge panels in Figures 1 and 2 should be read as statistically fragile comparisons rather than as resolved leaderboards. This is not merely a caveat. It is itself a benchmark finding: the merge regime is exactly where the offline–online crossover is most interesting, and also where the current reward design makes clean solver separation hardest to resolve. That combination is scientifically inconvenient, but substantively informative. The CI arithmetic also makes the next step concrete. Holding variance fixed at the current estimates, shrinking the best DrivingMerge row from about ± 7.70 to ± 7.5 would require about 317 episodes, while pushing it to about ± 5 would require roughly 712. For DrivingMergeNoisy, the current best row at about ± 8.53 would require about 388 episodes to reach ± 7.5 and about 873 to reach ± 5. In other words,

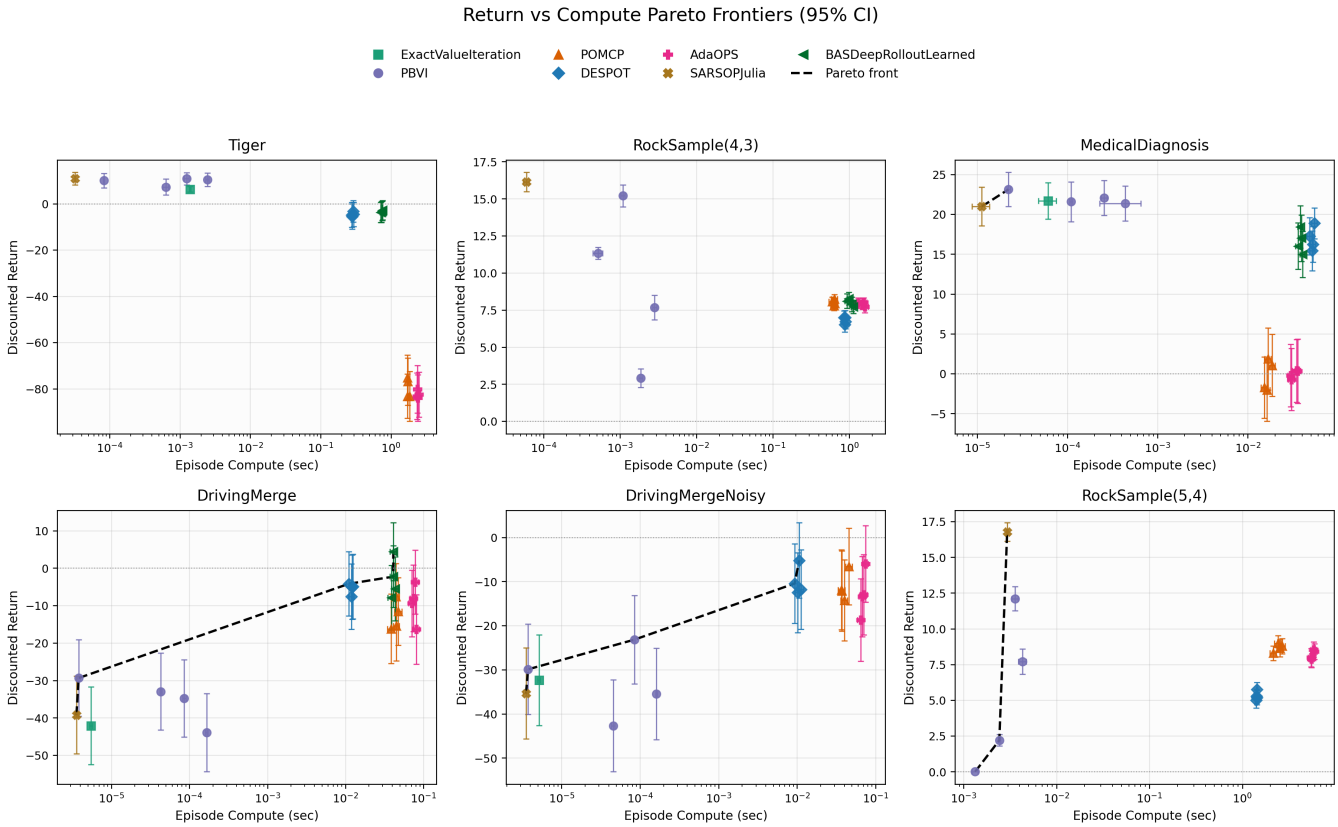

Figure 2: Return versus compute Pareto frontiers for the expanded 300-episode benchmark with `SARSOPJulia`. Compute is measured as episode-level online compute in seconds. Error bars show 95% confidence intervals, each panel uses a log compute axis, and dashed frontiers mark non-dominated solver–budget choices within an environment. The updated figure shows that offline alpha-vector methods dominate most of the tabular suite, while the merge tasks remain the clearest unresolved region in which online or learned search can become competitive.

even after 300 episodes, a truly tight merge comparison still means a few hundred more episodes, not a token extension.

5.5 Secondary Method Study: Learned `BASDeepRollout`

To test the compute-allocation view as an actual method rather than only as an explanatory lens, we evaluated a learned version of `BASDeepRollout` on the primary four-environment suite. The learned variant trains a compact policy–value checkpoint for each environment from teacher search traces using three rounds of dataset aggregation (100 total collection episodes at teacher budget 128), and then plugs that checkpoint into the same deep-allocation, rollout-gating, and leaf-evaluation machinery. We treat this as a secondary method study rather than as the paper's primary claim, because the method is intentionally evaluated as a diagnostic extension to the benchmark rather than as a fully tuned standalone solver family.

The one clearly positive BAS result is still DrivingMerge: PBVI remains strongly negative there, while the learned BAS planner reaches a positive mean return of 4.40 ± 7.70 at budget 128 and lies on or near the Pareto frontier. That is the regime in which adaptive online guidance is most plausible in the present suite. The broader diagnostic signal, however, is the shape of the scaling curve. `BASDeepRolloutLearned` remains fairly flat from budget 64 to 512 across the environments where checkpoints exist, which implies that the limiting factor is the learned representation rather than search depth itself: the planner is effectively getting similar guidance regardless of how many simulations it is allowed to run, so extra compute cannot recover what the model is not representing.

The environment-wise pattern is otherwise mixed but interpretable. On Tiger, the learned method improves dramatically over plain POMCP, but still remains below `SARSOPJulia`/PBVI and never becomes positive on average. On RockSample(4,3), it stays in roughly the same regime as POMCP and DESPOT but remains far behind the best offline alpha-vector methods. On MedicalDiagnosis, it is clearly viable and outperforms DESPOT and POMCP, yet PBVI still leads. The point is therefore not that learned BAS beats the offline baselines everywhere;

Table 2: Best observed mean return per environment from the 300-episode expanded run (reported as mean ± 95% confidence interval). Rows marked "Open" have 95% confidence intervals overlapping zero and therefore remain statistically unresolved at the current sample count.

Environment	Solver	Budget	Return ($\mu \pm \mathrm{CI}_{95}$)	Step ms ($\mu \pm \mathrm{CI}_{95}$)	Episode sec ($\mu \pm \mathrm{CI}_{95}$)	Status
Tiger	SARSOPJulia	64	10.88 ± 2.67	0.0022 ± 0.0000	0.000032 ± 0.000001	Resolve
RockSample(4,3)	SARSOPJulia	64	16.13 ± 0.65	0.0048 ± 0.0004	0.000060 ± 0.000005	Resolve
RockSample(5,4)	SARSOPJulia	64	16.77 ± 0.64	0.1787 ± 0.0032	0.002942 ± 0.000074	Resolve
DrivingMerge	BASDeepRolloutLearned	128	4.40 ± 7.70	14.2876 ± 0.6965	0.041099 ± 0.003806	Open
DrivingMergeNoisy	DESPOT	128	-5.23 ± 8.53	4.4440 ± 0.0161	0.010680 ± 0.000842	Open
MedicalDiagnosis	PBVI	512	23.12 ± 2.14	0.0082 ± 0.0002	0.000022 ± 0.000001	Resolve

Table 3: Representative offline precomputation times measured as a cold planner reset on the benchmark machine. These setup costs are not included in the online step/episode compute metrics reported elsewhere in the paper.

Environment	Solver (budget)	Sec
Tiger	ExactValueIteration (64)	0.58
Tiger	PBVI (256)	5.38
MedicalDiagnosis	ExactValueIteration (64)	7.05
MedicalDiagnosis	PBVI (128)	3.73
RockSample(4,3)	PBVI (512)	15.85

Table 4: Representative learned `BASDeepRollout` results. Entries report best mean return ± 95% confidence interval and episode-level online compute; "Best baseline" is the strongest non-BAS result from the same matched run.

Environment	Best learned BAS	Best baseline
RockSample (4,3)	Budget 128: 8.27 ± 0.43, 0.992 s	SARSOP-64: 16.13 ± 0.65, 0.000 s
DrivingMerge	Budget 128: 4.40 ± 7.70, 0.041 s	AdaOPS-256: −3.73 ± 8.53, 0.079 s
Medical Diagnosis	Budget 128: 18.42 ± 2.66, 0.038 s	PBVI-512: 23.12 ± 2.14, 0.000 s

it is that learned compute allocation begins to matter exactly where the structured offline baselines break down.

The expanded suite also makes the current scope of the BAS study more explicit: no learned checkpoints are yet available for RockSample(5,4) or DrivingMergeNoisy, so the learned-BAS comparison remains a four-environment diagnostic rather than a full six-environment benchmark competitor.

Two diagnostic patterns reinforce this reading. First, Tiger has the lowest policy KL (0.333), yet final control remains weak, which suggests that imitation fidelity alone is not enough when the teacher prior is itself a poor rollout guide. Second, RockSample(4,3) has the highest policy KL (0.955), which suggests that the compact learned model is missing a highly conditional action map even in a well-structured domain. Taken together with the flat scaling curves, these diagnostics point to model quality, not search budget, as the main current limitation.

We also ran a follow-up ablation that replaced the shared policy–value trunk with separate policy and value networks, switched optimization to Adam, and relabeled value targets between aggregation rounds with TD(λ) bootstrapping. The main effect was on value learning rather than across-the-board control: raw value MSE dropped sharply on RockSample(4,3) (about $28.6 \rightarrow 6.1$) and DrivingMerge (about $4261.9 \rightarrow 2084.3$), and the DrivingMerge learned-BAS point improved slightly while using slightly less online compute (6.92 ± 13.02 at 0.037 s $\rightarrow 7.40 \pm 13.11$ at 0.033 s). The gains were not uniform across environments, which again suggests that the remaining bottleneck is not only value-target variance but also policy-prior quality or belief-space coverage. Appendix Table 6 reports the corresponding environment-wise ablation summary.

The main methodological lesson is therefore narrower and stronger than a generic "BAS wins" claim. Dominance by `SARSOPJulia`/PBVI on Tiger and the RockSample domains is expected and not the interesting comparison. The real algorithmic signal is the DrivingMerge case, where a learned compute-allocation policy becomes competitive precisely when offline reusable structure weakens. Even there, the evidence is best read as supporting and diagnostic rather than as a standalone method victory.

5.6 Failure and Diagnostic Analysis

Tiger as a theory-of-failure case. Tiger is the cleanest example in the benchmark of a domain where stronger belief approximation or more online simulation does not rescue a mismatched solver. PBVI remains decisively positive, exact value iteration is competitive, and the online methods perform much worse despite substantially higher action-selection cost. The domain is small, sharply structured, and dominated by catastrophic wrong-door penalties, so a few misallocated search branches can destroy value. This makes Tiger less interesting as a difficulty benchmark than as a compact failure case for online Monte Carlo planning under severe asymmetric penalties.

Belief quality is diagnostic, not dispositive. The belief-divergence calculation is still useful, but mainly as a diagnostic rather than a ranking rule. Lower divergence usually means the particle approximation is becoming more faithful, yet the benchmark shows that accurate

belief tracking is only one ingredient of good control. When return saturates or degrades at larger budgets, the bottleneck appears to shift from belief approximation toward rollout quality, branching allocation, or the horizon at which simulation is effective. That is precisely why reporting reward, compute, and belief quality together is more informative than any one metric in isolation.

6 Discussion

6.1 Why Differences Occur

The cleanest mechanistic explanation in the paper still comes from Tiger. As discussed in the failure analysis above, Tiger combines tiny state support, highly reusable value structure, and catastrophic asymmetric penalties. That combination rewards solvers that can encode the policy geometry offline and punishes online Monte Carlo planners when even a small number of poorly allocated simulations overvalue the wrong door. More generally, the same pattern extends farther across the benchmark than the earlier draft suggested. With `SARSOPJulia` included, offline alpha-vector planning remains strongest not only on Tiger and MedicalDiagnosis but also on both RockSample variants, including RockSample(5,4). That means raw size or branching burden alone is not enough to explain the crossover. Structure helps when value information can be reused repeatedly, while online methods become attractive only when realized uncertainty is rich enough that reactive search can recover information unavailable to a fixed offline approximation. In that sense, observation structure, belief approximation, and branching effects matter not as abstract categories, but through the concrete way they shift the balance between reusable offline policy structure and per-decision online search.

The 300-episode rerun also highlights a second-order effect: increasing the sample count can change the confidence with which we tell the story, even when it does not dramatically change the winner table. The merge domains are the clearest example. Their mean rankings are informative, but the wide intervals show that these tasks are intrinsically volatile under the current reward design, so a careful paper should emphasize robustness and uncertainty rather than only the top row of the table.

DrivingMergeNoisy helps make that point concrete. Its best observed mean remains negative even after the larger rerun, which suggests a regime where extra online search is still not converting noisy observations into reliably decision-relevant signal. That makes it a useful reminder that moving toward online planning does not by itself guarantee strong control when the observation process becomes substantially more corrupted.

6.2 A Testable Crossover Conjecture

The budget-constrained policy mapping view from Section 2 suggests a structural conjecture for when offline and online methods should exchange dominance. Let $\mathcal{B}_{\text{reach}}(P)$ denote the reachable belief set of a POMDP, let $M(P)$ denote an effective measure of its reusable complexity (for example, covering number or approximation rank), and let $\chi(P)$ denote a branching-adjusted uncertainty index that grows with observation dispersion and effective lookahead burden. Informally, $M(P)$ is small when the belief geometry is compact and repeatedly reusable, while $\chi(P)$ is large when good decisions depend strongly on realized observation sequences.

Conjecture 1 (Structural crossover). For a fixed online compute allowance κ, there exists a threshold statistic

$$\Xi(P) = \frac{\chi(P)}{M(P)} \tag{12}$$

such that offline methods are favored when $\Xi(P)$ is small, while online simulation methods become favored when $\Xi(P)$ is large. Equivalently, compact reusable belief structure pushes the optimum of $\max_{\mathcal{S},B} R(\pi_B)$ s.t. $C(\pi_B) \leq \kappa$ toward offline policy construction, whereas high branching uncertainty pushes it toward adaptive online search.

This should be read as a descriptive hypothesis about regime ordering under fixed compute, not as a theorem and not as a claim that one scalar completely determines solver performance. The present benchmark offers partial support for that reading, but also stronger falsification pressure than before: Tiger and MedicalDiagnosis are low-Ξ domains in which offline methods dominate, yet even the higher-Ξ RockSample(5,4) instance remains offline-favoring once `SARSOPJulia` is included. The merge tasks are still the environments most directionally consistent with an online-favoring regime, but their wide confidence intervals mean that the benchmark currently locates the crossover region more confidently than it resolves it. That tension is not just a weakness; it is also part of the empirical picture. The most interesting regime boundary is precisely the one where uncertainty in solver ranking remains highest.

To make the conjecture testable, we also define a simple proxy version using only quantities available from the benchmark environments:

$$\widehat{M}(P) = \frac{1}{\log_2(1+|\mathcal{S}|)}, \qquad \widehat{\chi}(P) = H\,\log_2(1+|\mathcal{A}|)\,\bar{H}_O, \tag{13}$$

where H is the finite planning horizon stored in the benchmark environment definition (Tiger 15, MedicalDiagnosis 12, DrivingMerge 18, RockSample $\max(20, 4n+6)$), and $\bar{H}_O$ is the mean predictive observation entropy at the initial belief:

$$\bar{H}_O = \frac{1}{|\mathcal{A}|}\sum_{a\in\mathcal{A}}\left[-\sum_{o\in\mathcal{O}} p(o \mid b_0, a)\log_2 p(o \mid b_0, a)\right], \tag{14}$$

with

$$p(o \mid b_0, a) = \sum_{s'\in\mathcal{S}} Z(o \mid s', a)\sum_{s\in\mathcal{S}} T(s' \mid s, a) b_0(s). \tag{15}$$

In other words, $\bar{H}_O$ measures the average entropy, in bits, of the one-step predictive observation distribution induced by each action at the initial belief. This gives a coarse empirical score

$$\widehat{\Xi}(P) = \frac{\widehat{\chi}(P)}{\widehat{M}(P)}. \tag{16}$$

In the present environments, the resulting ordering is approximately Tiger (33), MedicalDiagnosis (41), DrivingMerge (75), DrivingMergeNoisy (77), RockSample(4,3) (127), and RockSample(5,4) (230). This proxy is intentionally crude. Its value is not that it fits every case, but that it is easy to compute and easy to falsify. The 300-episode benchmark makes its miss sharper than before: both RockSample instances are scored as harder than the merge tasks, yet strong offline alpha-vector methods remain best on them. That miss is informative because it points to exactly what the proxy fails to encode, namely reusable structure in the reachable belief geometry rather than raw state count alone.

One plausible mechanism is that $\widehat{\chi}(P)$ over-penalizes nominal branching in RockSample while $\widehat{M}(P)$ underestimates how compressible its reachable beliefs actually are. RockSample has more actions, observations, and local branching than the merge tasks, so the proxy assigns it a larger raw complexity score. But many of those branches collapse back onto reusable belief motifs: movement and checking actions repeatedly generate similar local information-gathering subproblems, and once a rock is resolved its downstream effect is simple. That kind of repeated reachable-belief geometry is exactly what strong offline alpha-vector methods can amortize well. By contrast, the merge tasks look smaller under the crude proxy, yet their useful beliefs may be less reusable because action quality depends on transient timing distinctions under noisy sensing. In that sense the proxy misfires for a principled reason: it sees nominal uncertainty growth, but not whether that uncertainty lives on a reusable low-dimensional belief manifold.

6.3 Practical Implications

Solver selection should be conditional on deployment constraints:

- If online latency is tight, precomputed offline policies are attractive.
- If environment mismatch and uncertainty are high, online methods can be safer despite compute cost.
- Budget sweeps are necessary because return and compute can decouple sharply.
- Benchmark evidence should be preferred over a single headline win, because solver rankings shift materially with environment structure.

6.4 Solver Selection by Regime

The benchmark also supports a more operational reading of the results. If the model class is trusted, the task will be solved many times, and online latency matters, strong offline alpha-vector methods are the best default in this suite: `SARSOPJulia` where available, and PBVI as the strongest fully internal approximate baseline. If the belief support becomes less reusable, online search becomes more compelling, but the merge tasks show that the choice among online solvers depends on what kind of compromise is acceptable. DESPOT often offers a more compute-efficient middle ground, AdaOPS can recover some value in the harder settings at substantial compute cost, and learned BAS becomes interesting precisely where PBVI degrades. This regime-based view is more useful than asking which solver is "best" in isolation.

Another practical lesson is that variance itself should be treated as part of the benchmark outcome. The 100-episode rerun improves confidence, but it does not erase the fact that some domains remain noisy enough to blur rankings. For practitioners, that means solver choice should not be made from mean return alone. A solver with slightly lower mean return but much lower online cost, tighter confidence intervals, or more stable belief tracking may be preferable in deployment.

The learned `BASDeepRollout` study adds one narrower lesson, but it remains secondary to the benchmark evidence. Its flat budget-response curves indicate that the current learned variant is limited more by representation quality than by search depth, and its clearest positive case is DrivingMerge, where PBVI degrades and adaptive online guidance starts to matter. The follow-up TD(λ) ablation is consistent with that reading: the biggest gains appear in the noisier long-horizon regime rather than in the small structured domains.

7 Limitations

- Environment suite is still modest in size and tabular, so the strongest claims should be read as benchmark evidence for a structured crossover pattern rather than as a comprehensive solver ranking.
- Baseline coverage is materially improved: a working Julia SARSOP bridge now gives a matched full-suite classical offline comparison on the tabular benchmark. The main remaining baseline gap is outside the current regime, especially the lack of a modern continuous-observation baseline such as POMCPOW.
- No deep-learning POMDP baselines are included.
- The learned `BASDeepRollout` study is incomplete on the harder pair of environments because learned checkpoints are not yet available for RockSample(5,4) or DrivingMergeNoisy.

- Belief divergence is only available for particle-based methods.
- Confidence intervals are reported from 300 episodes per setting, but the merge domains still exhibit wide intervals; under the current observed variances, reaching CI_{95} widths near ± 7.5 would still require roughly 317 episodes for the best DrivingMerge row and about 388 for the best DrivingMergeNoisy row, while widths near ± 5 would require roughly 712 and about 873, respectively, assuming comparable variance.
- Reported step-time and episode-compute metrics primarily reflect online action selection. Table 3 partially quantifies representative offline setup cost, but it does not exhaust the full wall-clock burden of all offline hyperparameter or planning choices.
- A materially stronger benchmark claim would require broader validation than is currently present: more environments beyond the tabular core, at least one continuous-observation setting with a POMCPOW-style comparison, and tighter resolution of the high-variance merge regimes.

8 Threats to Validity

Several threats to validity remain. In terms of external validity, the benchmark focuses on tabular and discretized environments, so the resulting rankings may not transfer directly to continuous-state or learned-model settings. In terms of construct validity, our compute metrics emphasize online latency and episode compute, which is the right lens for deployment-oriented comparison but understate large offline precomputation costs for methods such as PBVI or exact value iteration. In terms of internal validity, we mitigate implementation bias by using a shared harness, matched seeds, and identical environment definitions, but solver-specific hyperparameters, rollout policies, and approximation choices can still affect outcomes, especially for online Monte Carlo methods. Finally, coverage is not perfectly uniform across solvers: exact value iteration is intentionally capped on larger RockSample instances, and optional Julia-based components may depend on external tooling. We therefore view the benchmark as a reproducible comparison of practical solver regimes rather than as a definitive universal ranking.

9 Conclusion

We show that POMDP solver performance is strongly environment-dependent and tightly coupled to computational budget. The paper's central contribution is therefore benchmark-oriented rather than leaderboard-oriented: it exposes a structure-dependent crossover in which strong offline methods dominate where value structure is compact and reusable, and continue to dominate farther into the hard tabular regime than the earlier draft suggested. With `SARSOPJulia` included, that dominance now extends through both RockSample variants as well as Tiger and MedicalDiagnosis. The remaining plausible crossover region is the merge family, where offline methods degrade, learned or online methods become more competitive, and the rankings still remain statistically open even after 300 episodes. A single-score ranking is therefore misleading. Robust comparison requires joint reporting of return, uncertainty, belief quality, and compute. The benchmarking framework makes those trade-offs explicit and reproducible across both standard and custom uncertainty settings, and the proposed $\Xi(P)$ framing remains useful primarily as a falsifiable structural lens whose current failures point directly to missing reachable-belief geometry.

The learned `BASDeepRollout` study is best read as a supporting diagnostic rather than as the paper's central claim. Its main value in the present draft is to show that learned compute allocation becomes most plausible exactly in the regime where offline structure reuse weakens, while also revealing that the current learned instantiation is bottlenecked by representation quality rather than raw search budget. For the benchmark claim itself, the next validation step is now clearer: move beyond the current tabular core, add at least one continuous-observation setting with a POMCPOW-style comparison, complete learned-BAS coverage on the harder environments, and tighten the merge-task uncertainty with a few hundred more episodes rather than a token extension. Those upgrades would make the crossover story substantially more decisive while keeping the paper's main identity where its evidence is currently strongest.

A Best Solver at Each Budget

B Additional BAS Ablation

References

[1] Maximilian Igl, Luisa M. Zintgraf, Tuan Anh Le, Frank Wood, and Shimon Whiteson. Deep variational reinforcement learning for POMDPs. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[2] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.

[3] Péter Karkus, David Hsu, and Wee Sun Lee. Qmdpnet: Deep learning for planning under partial observability. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Table 5: Best observed solver at each tested belief budget for every environment in the 300-episode run. Rows marked as best are selected by highest mean discounted return among completed configurations.

Environment	Budget	Best solver	Return ($\mu \pm \mathrm{CI}_{95}$)	Episode sec ($\mu \pm \mathrm{CI}_{95}$)
Tiger	64	SARSOPJulia	10.88 ± 2.67	0.000032 ± 0.000001
Tiger	128	PBVI	10.82 ± 2.62	0.001250 ± 0.000101
Tiger	256	PBVI	10.44 ± 2.90	0.002465 ± 0.000135
Tiger	512	PBVI	10.00 ± 3.12	0.000083 ± 0.000001
RockSample(4,3)	64	SARSOPJulia	16.13 ± 0.65	0.000060 ± 0.000005
RockSample(4,3)	128	BASDeepRolloutLearned	8.27 ± 0.43	0.992003 ± 0.060600
RockSample(4,3)	256	POMCP	8.23 ± 0.32	0.636107 ± 0.058164
RockSample(4,3)	512	PBVI	15.20 ± 0.74	0.001088 ± 0.000063
RockSample(5,4)	64	SARSOPJulia	16.77 ± 0.64	0.002942 ± 0.000074
RockSample(5,4)	128	POMCP	8.78 ± 0.53	2.690148 ± 0.196578
RockSample(5,4)	256	PBVI	12.11 ± 0.84	0.003577 ± 0.000274
RockSample(5,4)	512	AdaOPS	8.54 ± 0.53	5.870800 ± 0.410959
DrivingMerge	64	BASDeepRolloutLearned	-2.29 ± 8.24	0.041085 ± 0.004016
DrivingMerge	128	BASDeepRolloutLearned	4.40 ± 7.70	0.041099 ± 0.003806
DrivingMerge	256	AdaOPS	-3.73 ± 8.53	0.078726 ± 0.007991
DrivingMerge	512	DESPOT	-4.92 ± 8.61	0.012511 ± 0.001155
DrivingMergeNoisy	64	DESPOT	-10.47 ± 8.99	0.009300 ± 0.000670
DrivingMergeNoisy	128	DESPOT	-5.23 ± 8.53	0.010680 ± 0.000842
DrivingMergeNoisy	256	DESPOT	-12.51 ± 9.05	0.010181 ± 0.000748
DrivingMergeNoisy	512	AdaOPS	-6.02 ± 8.67	0.074302 ± 0.006036
MedicalDiagnosis	64	ExactValueIteration	21.69 ± 2.30	0.000061 ± 0.000014
MedicalDiagnosis	128	PBVI	22.05 ± 2.20	0.000256 ± 0.000019
MedicalDiagnosis	256	PBVI	21.34 ± 2.18	0.000440 ± 0.000211
MedicalDiagnosis	512	PBVI	23.12 ± 2.14	0.000022 ± 0.000001

Table 6: TD(λ) ablation on learned `BASDeepRollout`. "Base" is the original learned policy–value setup; "TD(λ)" uses separate policy/value networks, Adam, and TD(λ) relabeling. Entries report the best learned-BAS point per environment from each run.

Env.	Variant	Best return	Ep. sec	Raw MSE
Tiger	Base	-0.72 ± 7.66	0.699	757.6
Tiger	TD(λ)	-2.21 ± 7.54	0.608	551.5
RockSample (4,3)	Base	8.05 ± 0.81	0.868	28.6
RockSample (4,3)	TD(λ)	7.20 ± 0.90	1.181	6.1
Medical Diagnosis	Base	20.65 ± 4.03	0.030	547.9
Medical Diagnosis	TD(λ)	17.81 ± 4.79	0.032	560.8
DrivingMerge	Base	6.92 ± 13.02	0.037	4261.9
DrivingMerge	TD(λ)	7.40 ± 13.11	0.033	2084.3

[4] Hanna Kurniawati, David Hsu, and Wee Sun Lee. Sarsop: Efficient point-based pomdp planning by approximating optimally reachable belief spaces. In *Robotics: Science and Systems (RSS)*, 2008.

[5] Michael L Littman, Anthony R Cassandra, and Leslie Pack Kaelbling. The witness algorithm: Solving partially observable markov decision processes. *Technical Report*, 1994.

[6] Yuanfu Luo, Haoyu Bai, David Hsu, and Wee Sun Lee. Importance sampling for online planning under uncertainty. *The International Journal of Robotics Research*, 38(2-3):162–183, 2019.

[7] Joelle Pineau, Geoffrey Gordon, and Sebastian Thrun. Point-based value iteration: An anytime algorithm for pomdps. In *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1025–1032, 2003.

[8] Stéphane Ross and Brahim Chaib-draa. Aems: An anytime online search algorithm for approximate policy refinement in large POMDPs. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2592–2598, 2007.

[9] Stéphane Ross, Joelle Pineau, Sébastien Paquet, and Brahim Chaib-draa. Online planning algorithms for POMDPs. *Journal of Artificial Intelligence Research*, 32:663–704, 2008.

[10] David Silver and Joel Veness. Monte-carlo planning in large pomdps. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 23, 2010.

[11] Richard D Smallwood and Edward J Sondik. The optimal control of partially observable markov processes over a finite horizon. *Operations Research*, 21 (5):1071–1088, 1973.

[12] Adhiraj Somani, Nan Ye, David Hsu, and Wee Sun Lee. Despot: Online pomdp planning with regularization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, 2013.

[13] Zachary Sunberg and Mykel J. Kochenderfer. Pomcpow: An online algorithm for POMDPs with continuous state, action, and observation spaces. *CoRR*, abs/1709.06196, 2017.